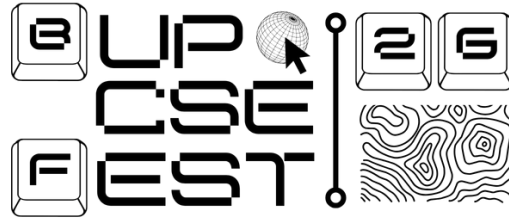




BUP COMPUTER PROGRAMMING CLUB



In association with



HACKATHON FINALS

BUP Fuel Supply Simulator

Integration and Interaction Guide

Scope: This document covers everything a participant application needs to **integrate** with the simulator and **interact** with its API surfaces (REST, SSE, admin). It does not cover challenge framing, deliverables, evaluation criteria, demo narrative, or strategy - see the official participant brief for those.

What is this simulator?

The **BUP Fuel Supply Simulator** is a deterministic, locally-runnable environment that models a small Bangladeshi fuel supply chain . There are two regions, two depots, four stations, six routes, three fuel types, and a 15-minute (controllable) tick clock. It exists so hackathon participants can build their own decision-making system **against a realistic, reproducible world** instead of guessing how a real supply chain behaves.

Why does it exist?

The hackathon challenges participants to design a system that **moves fuel from depots to stations** while keeping stations stocked and customers happy. Real supply chains are noisy, slow, and unfair to compare against. This simulator gives every participant the **same world with the same demand curve, the same initial inventory, and the same injected crises**, so judges can fairly score their decisions rather than their luck.

How do I integrate my system with it?

Your application runs **outside** the simulator container and talks to it over HTTP. You **read** the world through REST endpoints (/v1/instance, /v1/depots, /v1/stations, /v1/routes, /v1/supply-arrivals, /v1/demand-history, /v1/metrics, /v1/allocations), **listen** for change notifications via Server-Sent Events (/v1/stream), and **write** your fuel-replenishment decisions through exactly one endpoint : **POST /v1/allocations**. The simulator then runs those decisions against the world, advances time, and reports back what happened. Use the historical and simulated data available through these interfaces to build predictive capabilities and continuously improve your decisions. Treat REST as the source of truth; treat SSE as a hint that something changed.

Table of Contents

1. Fast start
2. Hard rules
3. Quick reference card
4. Public /v1/* endpoints
5. POST /v1/allocations (the only domain write)
6. SSE stream /v1/stream
7. Admin /admin/* endpoints
8. The simulated world
9. Status-code cheat sheet
10. Defensive client checklist

1. Fast start

The simulator ships as a single docker-compose.yml:

```
services:
  simulator-api:
    image: asifmahmoud414/bup-fuel-supply-simulator:1.0.0
    environment:
      SIMULATION_SPEED: ${SIMULATION_SPEED:-8}
      TICK_MINUTES: ${TICK_MINUTES:-15}
      SIMULATOR_START_MODE: ${SIMULATOR_START_MODE:-paused} # Options: paused, running
    ports:
      - "8000:8000"
```

SIMULATION_SPEED : Wall-clock ticks per second while running. Higher = faster time.

TICK_MINUTES : Simulated minutes advanced per tick. Larger = coarser steps.

SIMULATOR_START_MODE : Start ticking immediately (running) or stay paused until resumed.

Start the stack with:

```
docker compose up -d
curl -s http://localhost:8000/v1/health
```

2. Hard rules

- **The simulator is the world, not the brain.** It does not predict, optimize, recommend, or decide. It executes your POST /v1/allocations calls deterministically and reports what happened.
- **REST is the source of truth.** Treat SSE as a notification stream — always re-GET the affected resource after every event.
- **The world is deterministic.** Same scenario + same seed + same actions + same injected events ⇒ byte-identical state, including per-tick demand jitter.
- **The scenario is baked into the image.** The active scenario cannot be switched at runtime. The default scenario has no preloaded events.
- **One simulator instance per participant.** Single-tenant local. There is no central server.
- **Do not modify the simulator source** to solve the challenge. Judges run your submission against the published image.
- **/admin/* paths bypass all fault injection.** To test fault handling, hit /v1/* paths only. /v1/health also bypasses faults (use as liveness probe).
- **Only allocations can be written from /v1/*.** Every other resource is read-only. Crisis events and faults are [injected via /admin/*](#) (your app can call them too for self-test scenarios).

3. Quick reference card

Setting	Details
Base URL (local)	http://localhost:8000
Swagger UI	http://localhost:8000/docs
ReDoc	http://localhost:8000/redoc

Simulator Dashboard	http://localhost:8000/admin
Idempotency-Key	Not in headers. Sent as idempotency_key body field on /v1/allocations.
Stale data signal	X-Simulator-Stale: true response header on /v1/* GETs when a stale-data fault is active.
Default tick	15 simulated minutes per tick (TICK_MINUTES env var)
Default speed	8 ticks/sec wall-clock while RUNNING (SIMULATION_SPEED)

4. Public /v1/* Endpoints

Every endpoint below returns JSON unless noted. All /v1/* endpoints except /v1/health are subject to fault injection; /admin/* are not.

4.1 GET /v1/health

Bypasses faults. Use as your liveness probe.

```
{
  "status": "ok",
  "database": "ok",
  "simulation": { "status": "PAUSED", "tick": 0 }
}
```

4.2 GET /v1/instance

The simulation instance (there is only one). Read often for current tick, sim_time, and status.

```
{
  "id": 1,
  "scenario_id": "baseline",
  "scenario_version": "1.0",
  "seed": 12345,
  "sim_time": "2026-01-01T00:00:00+00:00",
  "tick": 0,
  "tick_minutes": 15,
  "status": "PAUSED"
}
```

status : {"PAUSED", "RUNNING"}.

4.4 GET /v1/regions

```
[
  { "id": "region-dhaka",      "name": "Dhaka Division",      "demand_factor": 1.00 },
  { "id": "region-chattogram", "name": "Chattogram Division", "demand_factor": 1.08 }
]
```

4.5 GET /v1/depots and GET /v1/depots/{entity_id}

```
[
  {
    "id": "depot-gazipur",
    "name": "Gazipur Depot",
    "region_id": "region-dhaka",
    "status": "OPEN",
  }
]
```

```

    "dispatch_capacity_per_tick": 12000,
    "capacity": { "DIESEL": 90000, "PETROL": 70000, "OCTANE": 45000 },
    "inventory": { "DIESEL": 60000, "PETROL": 45000, "OCTANE": 26000 }
  }
]

```

depot.status: {"OPEN", "CONSTRAINED"}. 404 {"detail":{"code":"NOT_FOUND"}} if id is unknown.

4.6 GET /v1/stations and GET /v1/stations/{entity_id}

```

[
  {
    "id": "station-mirpur",
    "name": "Mirpur Fuel Station",
    "region_id": "region-dhaka",
    "status": "OPEN",
    "demand_profile": "urban_high",
    "demand_multiplier": 1.0,
    "capacity": { "DIESEL": 15000, "PETROL": 14000, "OCTANE": 9000 },
    "inventory": { "DIESEL": 9000, "PETROL": 9000, "OCTANE": 5000 }
  }
]

```

station.status : {"OPEN", "OUTAGE"}. demand_multiplier is mutated at runtime by demand_spike events.

4.7 GET /v1/routes

```

[
  {
    "id": "route-gazipur-mirpur",
    "source_depot_id": "depot-gazipur",
    "destination_station_id": "station-mirpur",
    "transit_ticks": 2,
    "max_shipment": 7000,
    "status": "AVAILABLE"
  }
]

```

route.status : {"AVAILABLE", "DISRUPTED"}.

4.8 GET /v1/supply-arrivals

Sorted by planned_tick ascending.

```

[
  {
    "id": "supply-001",
    "depot_id": "depot-gazipur",
    "fuel_type": "DIESEL",
    "quantity": 18000,
    "planned_tick": 12,
    "actual_tick": null,
    "status": "SCHEDULED"
  }
]

```

status : {"SCHEDULED", "DELAYED", "ARRIVED"}.

4.9 GET /v1/events

Domain events (injected by you or preloaded by the scenario file), id-desc.

```
[
  {
    "id": 1,
    "type": "demand_spike",
    "start_tick": 8,
    "end_tick": 20,
    "status": "RESOLVED",
    "parameters": { "region_ids": ["region-dhaka"], "multiplier": 1.8 }
  }
]
```

status : {"SCHEDULED", "ACTIVE", "RESOLVED"}.

4.10 GET /v1/allocations

Sorted id-desc. Every allocation your app has created (and any preloaded ones).

```
[
  {
    "id": 1,
    "idempotency_key": "demo-001",
    "source_depot_id": "depot-gazipur",
    "destination_station_id": "station-mirpur",
    "route_id": "route-gazipur-mirpur",
    "fuel_type": "DIESEL",
    "quantity": 3000,
    "created_tick": 5,
    "departure_tick": 6,
    "expected_arrival_tick": 8,
    "actual_arrival_tick": 8,
    "status": "ARRIVED",
    "failure_reason": null
  }
]
```

status : {"PENDING", "IN_TRANSIT", "ARRIVED", "FAILED", "CANCELLED"}.

4.11 GET /v1/demand-history?station_id=&limit=

Time-series of demand observations. **limit is clamped to [1, 2000], default 200**. The table grows unboundedly — always call with a limit and filter to recent N.

```
[
  {
    "id": 100,
    "station_id": "station-mirpur",
    "fuel_type": "DIESEL",
    "tick": 12,
    "sim_time": "2026-01-01T03:00:00+00:00",
    "demand_liters": 95.123,
    "served_liters": 95.123,
    "unmet_liters": 0.0
  }
]
```

```
] ]
```

A row is inserted per (station_id, fuel_type) per tick -> 4 stations × 3 fuels = 12 rows per tick.

4.12 GET /v1/metrics

Aggregated ground-truth metrics since simulation start.

```
{
  "served_demand_liters": 12345.678,
  "unmet_demand_liters":   234.567,
  "service_level":         0.981408,
  "allocation_liters":     9800.000,
  "allocation_failures":   2
}
```

- **service_level** = served / (served + unmet) — **1.0** means zero unmet demand.
- **allocation_liters** counts only IN_TRANSIT + ARRIVED shipments (not PENDING, FAILED, CANCELLED).
- **allocation_failures** is the count of status="FAILED" allocations (route disruption at departure time).

5. POST /v1/allocations — the only domain write

This is the **only** domain-mutating endpoint participants should call from /v1/*.

5.1 Request

```
POST /v1/allocations
Content-Type: application/json
```

```
{
  "idempotency_key": "demo-001",
  "source_depot_id": "depot-gazipur",
  "destination_station_id": "station-mirpur",
  "route_id": "route-gazipur-mirpur",
  "fuel_type": "DIESEL",
  "quantity": 3000
}
```

Field	Type	Required	Constraints
idempotency_key	string	yes	length 1–150, unique unless replaying the exact same request
source_depot_id	string	yes	must exist in /v1/depots
destination_station_id	string	yes	must exist in /v1/stations
route_id	string	yes	must exist in /v1/routes
fuel_type	enum	yes	DIESEL PETROL OCTANE
quantity	float	yes	> 0, ≤ route.max_shipment

5.2 Validation order (first failure wins)

- **Idempotency check** — see 5.5.

- **NOT_FOUND (404)** if depot, station, or route id is unknown.
- **ROUTE_MISMATCH (409)** if the route's (source_depot_id, destination_station_id) ≠ the ones in your request.
- **DEPOT_CLOSED (409)** if depot.status is not in {"OPEN", "CONSTRAINED"}.
- **STATION_CLOSED (409)** if station.status != "OPEN".
- **ROUTE_DISRUPTED (409)** if route.status != "AVAILABLE".
- **ROUTE_CAPACITY_EXCEEDED (409)** if quantity > route.max_shipment.
- **INSUFFICIENT_INVENTORY (409)** if depot.inventory[fuel] < quantity.
- **DISPATCH_CAPACITY_EXCEEDED (409)** if (sum of in-flight + pending quantities from this depot on this tick) + quantity > depot.dispatch_capacity_per_tick.
- **DESTINATION_CAPACITY_EXCEEDED (409)** if station.inventory[fuel] + quantity > station.capacity[fuel].

5.3 Success response (201 Created)

```
{
  "id": 1,
  "idempotency_key": "demo-001",
  "source_depot_id": "depot-gazipur",
  "destination_station_id": "station-mirpur",
  "route_id": "route-gazipur-mirpur",
  "fuel_type": "DIESEL",
  "quantity": 3000,
  "created_tick": 5,
  "departure_tick": null,
  "expected_arrival_tick": null,
  "actual_arrival_tick": null,
  "status": "PENDING",
  "failure_reason": null
}
```

5.4 Idempotency rules

- **Same key, same body** → return existing allocation (HTTP 201, not 200). Safe retry.
- **Same key, different body** → 409 **IDEMPOTENCY_KEY_MISMATCH**. The first submission wins; you must use a new key.
- **Cancellation does NOT free the key**. Once used, an idempotency_key is permanently occupied.

5.5 POST /v1/allocations/{allocation_id}/cancel

Refunds the depot inventory and marks the allocation CANCELLED. Only valid for status="PENDING".

- **200 OK** with the updated allocation.
- **404 ALLOCATION_NOT_FOUND** if id unknown.
- **409 CANNOT_CANCEL** if status ≠ PENDING.

6. SSE stream /v1/stream

GET /v1/stream returns text/event-stream. It is the only way to receive push notifications from the simulator.

6.1 Wire protocol

- **On connect:** server sends : connected\n\n (SSE comment, ignore).

- **Each event:** event: <name>\ndata: <json>\n\n
- **After 15 seconds of silence:** : keepalive\n\n.
- **Buffer:** asyncio.Queue(maxsize=200) per subscriber. **If you fall behind by >200 events, your queue is silently dropped** and the stream will keep flowing to other subscribers. You must reconnect.

6.2 Reconnection

There is no Last-Event-ID replay. On reconnect you get : connected\n\n and then only events from that moment forward. Always re-fetch your state via REST after a reconnect.

6.3 Event names + payloads

Event name	When	Payload
simulation.tick	End of every tick	{"tick": <int>, "sim_time": "<ISO-8601>"}
allocation.status_changed	Allocation created / departed / arrived / failed / cancelled	serialize(allocation) — full allocation object
inventory.updated	Depot inventory changed	{"entity_type":"depot","entity_id":"<id>","inventory":{"...}}
simulator.notice	POST /admin/reset, background-runner exception	{"message":"..."} or {"level":"error","message":"..."}

Audit-log action values like event.started, event.resolved, supply.arrived, allocation.departed, allocation.arrived, simulation.tick are not SSE event names — read them via GET /admin/audit.

6.4 Fault interaction

- If a stream_disconnect fault is active, **GET /v1/stream returns 503** {"detail":{"code":"FAULT_INJECTED"}} instead of opening the stream.
- If a stale_data fault is active, **non-stream /v1/* GET responses include X-Simulator-Stale: true**. The SSE stream itself does not.

7. Admin /admin/* endpoints

All /admin/* paths **bypass fault injection**. Designed for organizer use, but participants will find them invaluable for self-testing (injecting events, advancing ticks deterministically, clearing faults).

7.1 GET /admin

Server-rendered HTML console. Auto-refreshes every 2 seconds. Shows live instance state, metrics, row counts, recent audit log, and forms to inject events / faults and run/pause/step/reset the simulation. Useful for visual debugging.

7.2 POST /admin/run — start the simulation

Sets SimulationInstance.status = "RUNNING". The background runner begins ticking.

7.3 POST /admin/pause — pause the simulation

Sets SimulationInstance.status = "PAUSED". The background runner idles; existing PENDING / IN_TRANSIT allocations stay frozen.

7.4 POST /admin/toggle — flip RUNNING ↔ PAUSED

Convenience for UIs.

7.5 POST /admin/step — deterministic single-tick advance

Calls tick_once() exactly once. Works regardless of status (you can step while PAUSED). Returns:

```
{ "tick": 6, "sim_time": "2026-01-01T01:30:00+00:00" }
```

This is the recommended way to drive deterministic tests. Pause, run your allocations, then call /admin/step a known number of times for reproducible outcomes.

7.6 POST /admin/reset — wipe everything and reload the active scenario

Hard reset. Deletes every row from demand_observations, allocations, events, faults, audit_logs, supply_arrivals, routes, stations, depots, regions, and simulation_instances, then reloads from the baked scenario YAML. Publishes simulator.notice {message: "Simulation reset"}.

7.7 POST /admin/events — inject a crisis event

POST /admin/events
Content-Type: application/json

```
{
  "type": "demand_spike",
  "start_tick": 8,
  "duration_ticks": 12,
  "parameters": { "region_ids": ["region-dhaka"], "multiplier": 1.8 }
}
```

Field	Type	Required	Constraints
type	enum	yes	one of six values (see §7.8)
start_tick	int	yes	>= 0
duration_ticks	int	yes	> 0
parameters	object	no	defaults to {}

end_tick = start_tick + duration_ticks. Returns 201 Created with the persisted event.

7.8 Event-type parameter shapes

type	Parameters	Effect while ACTIVE	Reversed on RESOLVE
------	------------	---------------------	---------------------

demand_spike	multiplier (def 1.5), station_ids[], region_ids[]	multiplies each affected station's demand_multiplier by multiplier	divides by multiplier (≥ 0.01)
route_disruption	route_ids[]	sets each route → DISRUPTED	routes → AVAILABLE
station_outage	station_ids[]	sets each station → OUTAGE (served drops to 0)	stations → OPEN
depot_constraint	depot_ids[]	sets each depot → CONSTRAINED (still shippable, but signals reduced capacity)	depots → OPEN
shipment_delay	delay_ticks (def 2), depot_ids[], fuel_types[]	SupplyArrival.planned_tick += delay_ticks, status → DELAYED	one-shot — does not auto-undo
supply_shortfall	factor (def 0.5), depot_ids[], fuel_types[]	SupplyArrival.quantity *= factor	one-shot — does not auto-restore

station_ids / region_ids / route_ids / depot_ids are filters: if the list is empty, the event applies to all entities of that type. fuel_types works the same way.

7.9 POST /admin/faults — inject a fault

POST /admin/faults
Content-Type: application/json

```
{
  "type": "stale_data",
  "duration_seconds": 60,
  "parameters": {}
}
```

Field	Type	Required	Constraints
type	enum	yes	one of five values (see §7.10)
duration_seconds	int	yes	$0 < x \leq 3600$
parameters	object	no	defaults to {}

start_wall_time = now(), end_wall_time = now() + duration_seconds, active=true. Auto-expires — no need to clear manually unless you want to.

7.10 Fault-type effects

type	parameters	Effect on /v1/* (not /admin/*, not /v1/health)
latency	{ "delay_ms": 500 } (default 500)	sleeps delay_ms / 1000 seconds before every non-admin, non-health request
unavailable	—	returns 503 { "error": { "code": "FAULT_INJECTED", "message": "Simulator API temporarily unavailable." } }

error_rate	{ "rate": 0.25 } (default 0.25)	with probability rate, returns 503 { "error": { "code": "FAULT_INJECTED", "message": "Injected transient API error." } }
stale_data	—	does NOT block requests; adds X-Simulator-State: true to /v1/* GET responses
stream_disconnect	—	GET /v1/stream returns 503 { "detail": { "code": "FAULT_INJECTED" } } (note: nested under detail, not error)

7.11 POST /admin/faults/clear

Sets active = false on every active Fault. Idempotent. Returns { "status": "cleared" }.

7.12 GET /admin/audit?limit=

limit clamped to [1, 1000], default 200. Sorted id-desc. This is the ground-truth audit log of everything the simulator has done.

```
[
  {
    "id": 142,
    "wall_time": "2026-01-01T00:15:00.512345+00:00",
    "sim_time": "2026-01-01T00:14:00+00:00",
    "tick": 0,
    "action": "allocation.created",
    "entity_type": "allocation",
    "entity_id": "1",
    "result": "OK",
    "metadata_json": { "quantity": 3000, "fuel_type": "DIESEL" }
  }
]
```

Useful action values: simulation.tick, allocation.created, allocation.departed, allocation.arrived, allocation.cancelled, event.created, event.started, event.resolved, supply.arrived, admin.run, admin.pause, fault.created, fault.clear_all.

7.13 GET /admin/faults and GET /admin/events

Last 50 rows each, id-desc. Useful for visualizing the fault/event timeline.

8. The simulated world

The active scenario is baked into the image. The shipped image runs baseline.yaml (no preloaded events). All scenarios share the same fixed world below; they differ only in seed and preloaded events.

8.1 Regions (2)

id	name	demand_factor
region-dhaka	Dhaka Division	1.00
region-chattogram	Chattogram Division	1.08

8.2 Depots (2)

id	region	dispatch / tick	capacity (D/P/O)	initial inventory (D/P/O)
depot-gazipur	region-dhaka	12,000	90,000 / 70,000 / 45,000	60,000 / 45,000 / 26,000
depot-patiya	region-chattogram	11,000	85,000 / 65,000 / 40,000	55,000 / 42,000 / 24,000

8.3 Stations (4)

id	region	demand_profile	capacity (D/P/O)	initial inventory (D/P/O)
station-mirpur	region-dhaka	urban_high	15,000 / 14,000 / 9,000	9,000 / 9,000 / 5,000
station-tongi	region-dhaka	industrial	18,000 / 9,000 / 6,000	11,000 / 6,000 / 3,500
station-karnaphuli	region-chattogram	highway	14,000 / 15,000 / 9,000	8,500 / 9,500 / 5,200
station-coxsbazar	region-chattogram	regional	12,000 / 12,000 / 7,000	7,500 / 7,500 / 4,200

8.4 Routes (6)

id	depot	station	transit_ticks	max_shipment
route-gazipur-mirpur	depot-gazipur	station-mirpur	2	7,000
route-gazipur-tongi	depot-gazipur	station-tongi	2	6,500
route-patiya-karnaphuli	depot-patiya	station-karnaphuli	2	7,000
route-patiya-coxsbazar	depot-patiya	station-coxsbazar	3	6,000
route-gazipur-karnaphuli	depot-gazipur	station-karnaphuli	4	5,000
route-patiya-mirpur	depot-patiya	station-mirpur	4	5,000

8.5 Demand profiles (liters per simulated day)

profile	DIESEL	PETROL	OCTANE	noise
urban_high	8,500	10,500	5,600	0.10
industrial	14,000	4,500	2,200	0.08
highway	10,500	11,000	6,200	0.12
regional	7,200	7,600	3,600	0.10

8.6 Hour-of-day factors

profile	busy hours (factor)	off-peak hours (factor)
---------	---------------------	-------------------------

industrial	06:00–17:59 → 1.55	18:00–05:59 → 0.45
highway	06–09 or 16–20 → 1.35	else → 0.75
urban_high	07–09 or 16–20 → 1.45	else → 0.70
regional	07:00–20:59 → 1.25	21:00–06:59 → 0.65

8.7 Supply arrival pattern

All scenarios share the same 22-arrival schedule:

- 4 "initial burst" arrivals at ticks 12–20 to cover day 1.
- 18 "recurring resupply" arrivals spaced 64 ticks apart (~16 simulated hours), top-up quantities sized to roughly one day's regional demand.

9. Status-code cheat sheet

All `/v1/allocations` errors come back as `{"detail": {"code": "<UPPER_SNAKE>", "message": "..."} }`. Injected faults return `{"error": {"code": "FAULT_INJECTED", "message": "..."} }` (note the difference: error vs detail). Pydantic validation errors use FastAPI's default `{"detail": [...] }`.

HTTP	Code	Trigger	Recommended handling
200	—	Idempotent replay of an existing allocation	Continue normally.
201	—	New allocation accepted	Cache the returned id.
404	NOT_FOUND	Depot, station, or route id unknown	Check your ids against <code>/v1/depots / /v1/stations / /v1/routes</code> .
404	ALLOCATION_NOT_FOUND	Cancel on unknown allocation id	Check <code>/v1/allocations</code> for the current id.
409	IDEMPOTENCY_KEY_MISMATCH	Same idempotency_key, different body	Use a new key for the new request; the first wins.
409	ROUTE_MISMATCH	Route connects different endpoints than your request	Use a route that matches your (depot, station) pair.
409	DEPOT_CLOSED	Depot status not in {OPEN, CONSTRAINED}	Wait for depot_constraint event to resolve.
409	STATION_CLOSED	Station status ≠ OPEN	Wait for station_outage event to resolve.
409	ROUTE_DISRUPTED	Route status ≠ AVAILABLE	Wait for route_disruption event to resolve, or pick another route.
409	ROUTE_CAPACITY_EXCEEDED	quantity > route.max_shipment	Split into multiple smaller allocations.
409	INSUFFICIENT_INVENTORY	Depot doesn't have the fuel	Re-fetch depot inventory; wait for next supply arrival.
409	DISPATCH_CAPACITY_EXCEEDED	Sum of this depot's in-flight + pending on this tick > dispatch_capacity_per_tick	Wait for the next tick (PENDING allocations free up).
409	DESTINATION_CAPACITY_EXCEEDED	station.inventory[fuel] + quantity > station.capacity[fuel]	Wait for station demand to consume fuel.

409	CANNOT_CANCEL	Cancel on non-PENDING allocation	The shipment is already in motion; you cannot recover it.
422	(Pydantic)	Bad enum, missing field, duration_ticks=0, quantity ≤ 0, etc.	Validate your payload client-side first.
503	FAULT_INJECTED	Active unavailable / error_rate / stream_disconnect fault	Implement backoff + retry + degraded mode. See §10.

10. Defensive client checklist

Every robust participant client should handle all of these gracefully:

```

GET /v1/health      - liveness check (bypasses faults)
GET /v1/instance    - current tick, sim_time, status, seed
GET /v1/depots       - capacity, inventory, dispatch_capacity_per_tick
GET /v1/stations     - inventory, demand_multiplier, capacity
GET /v1/routes       - availability, max_shipment, transit_ticks
GET /v1/supply-arrivals - upcoming deliveries
GET /v1/events       - past/active/scheduled crisis events
GET /v1/allocations  - your shipment ledger
GET /v1/demand-history - time-series for forecasting (limit 1-2000)
GET /v1/metrics      - service_level + allocation_failures
POST /v1/allocations - submit shipment, returns 201/200/404/409/503
POST /v1/allocations/{id}/cancel - refunds PENDING only
GET /v1/stream       - SSE feed (queue size 200; 15s keepalive)

```

SSE rules:

1. Treat SSE as advisory — re-GET REST state after every interesting event.
2. Watch X-Simulator-State: true on /v1/* GETs and invalidate any local cache.
3. If a stream_disconnect fault is active, GET /v1/stream returns 503 — back off and retry.
4. If your queue falls >200 events behind, you will be silently dropped. Reconnect and refetch state.
5. **There is no Last-Event-ID replay.** On reconnect, you only get events from that moment forward.
6. A 15-second silence is normal (keepalive); do not treat it as a disconnect.